Design document for calendar system in C

1. Overview:
   In the code, I use a linked list for storing venues. The events of each venue are also stored in a linked list, separately for each date. The linked list of events is sorted by time i.e. [start time, end time]. Every time an insertion or a deletion of events happens, I ensure that this sorted order is maintained because the showEvents command expects the events to be shown in chronological order.
   The reason of using a linked list to store the venues and events is because of its efficient addition and deletion properties. Also the events need to be printed chronologically, so using a linked list of events, each containing the interval of start and end time makes it very easy to sort the events according to time.

2. Data structures used:
   1. Struct event:
      This structure is for an individual event in a venue. The struct event includes the following fields: char name[] stores the name of the event, int s represents the start time (ranging from 0 to 23), int e denotes the end time (also ranging from 0 to 23), and struct event *next is a pointer to the next event in the linked list for the same date, allowing events to be organized in chronological order.
   2. Struct venue:
      Represents a venue where events are held. The `struct venue` includes the following fields: `char name[]` stores the name of the venue, `char loc[]` contains the venue's location, and `char cap[]` represents the venue's capacity. The `int totalEvents` field tracks the total number of events scheduled at the venue. Additionally, `struct event *date[30]` is an array of pointers to linked lists of events, with each index representing a day of the month, while `int eventsOnDate[30]` keeps track of the number of events scheduled on each specific day.
      The Venue List Head (`struct venue *head`) serves as the head pointer for the singly linked list of venues, enabling easy traversal and management of multiple venues.

3. Other implementation details:

Input: I use fgets() to take the input. Then I must separate the first word of the user's input and get the command given. I'm using many helper functions to accomplish this task. For implementing each command, there is a special function dedicated to it.

Error Handling: The system provides error messages for conditions such as adding events to non-existent venues, scheduling conflicts, or deleting non-existent entities.